FitText: Evolving Agent Tool Ecologies via Memetic Retrieval

Kyle Zheng*
UCLA
kylezheng@g.ucla.edu

Han Zhang*
UCLA
hhzhang@cs.ucla.edu

Renliang Sun
UCLA
sunrenliang@ucla.edu

Chenchen Ye
UCLA
ccye@cs.ucla.edu

Wei Wang
UCLA
weiwang@cs.ucla.edu

Abstract

A semantic gap separates how users describe tasks from how tools are documented. As API ecosystems scale to tens of thousands of endpoints, static retrieval from the initial query alone cannot bridge this gap: the agent's understanding of what it needs evolves during execution, but its tool set does not. We introduce `FitText`, a training-free framework that makes retrieval dynamic by embedding it directly in the agent's reasoning loop. `FitText` generates natural-language pseudo-tool descriptions as retrieval probes, refines them iteratively using retrieval feedback, and explores diverse alternatives through stochastic generation. Memetic Retrieval adds evolutionary selection pressure over candidate descriptions, guided by a tool memory that avoids redundant search. On ToolRet (43k tools, 4 domains), `FitText` improves average retrieval rank from 8.81 to 2.78; on StableToolBench (16,464 APIs), it achieves a 0.73 average pass rate—a 24-point absolute gain over static query retrieval. The gains transfer across base models capable of acting as competent semantic operators; under weaker base models, Memetic's evolutionary search inverts—amplifying noise rather than refining signal—surfacing model capacity as a prerequisite for evolutionary tool exploration.

1 Introduction

Large language models (LLMs) demonstrate strong reasoning and planning capabilities (Minaee et al., 2024), yet remain limited in their ability to interact with the external world through tools and APIs. To overcome this limitation, tool-augmented LLM agents have been introduced, enabling models to invoke APIs, databases, and external environments for more complex and grounded task execution (Qin et al., 2024a; Qu et al., 2025).

A fundamental challenge in such systems lies in identifying which tools should be used for a given task, a process known as **tool retrieval**. Existing approaches fall into two paradigms, both with critical limitations: **(i) Schema injection**, which provides agents with a manually curated, fixed set of candidate tools in the prompt context (Huang et al., 2023; Shen et al., 2023; Paranjape et al., 2023) causing context rot, as assumed by standard benchmarks (Guo et al., 2024; Qin et al., 2024b). Furthermore, manual tool curation is unrealistic as tool ecosystems grow to tens of thousands of APIs (Qin et al., 2024b). **(ii) Static root-query retrieval**, which selects tools once before execution based on the initial user query (Qu et al., 2024b). However, user intent is inherently amorphous and a query alone may not fully capture information needs (Asai et al., 2023), making one-shot retrieval brittle. Both paradigms assume that all required tools can be determined upfront, yet agents often discover the need for new tools *during* reasoning as their understanding of the task evolves. This motivates **dynamic retrieval**, where tool discovery is integrated into the reasoning

*Equal contribution. Correspondence to kylezheng@g.ucla.edu.

process, allowing the agent to decide *when* and *what* to retrieve adaptively throughout execution.

Compounding the problem, prior retrieval methods overlook the substantial *semantic gap* between user queries and tool descriptions. Queries are high-level and goal-oriented (e.g., Figure 1, "My flight was cancelled, help me get a refund"), while tool descriptions are functional (e.g., "Get the current status of a flight"). This gap is further widened in stochastic API environments whose available functionality cannot be anticipated at query time, and by general-purpose embedding models not trained for cross-domain matching between task intent and tool metadata.

To address these challenges, we propose `FitText`, an LLM-guided dynamic tool retrieval framework (Figure 1). Instead of using the user query directly, the LLM reasons about the task and produces a *pseudo-tool description*, a natural-language hypothesis of the needed tool's functionality. This description bridges the semantic gap between user intent and tool metadata, enabling retrieval in a semantically aligned space.

This parallels pseudo-answer rewriting and query rewriting in RAG (Li et al., 2024; Liu & Mozafari, 2024), where generating a hypothetical answer bridges the query–document gap; our pseudo-tool descriptions analogously bridge the query–tool gap. While sharing conceptual roots with hypothetical document generation (Gao et al., 2022; Wang et al., 2023) and pseudo-relevance feedback (Lavrenko & Croft, 2001), tool retrieval introduces distinct challenges: tools are structured with parameters and schemas, retrieved tools must be *executable*, and the agent's evolving context provides a richer refinement signal than a static query. We also adapt corpus-steered query expansion (Lei et al., 2024) to the tool domain.

Our main contributions are summarized as follows:

- We introduce `FitText`, a training-free framework that reframes tool retrieval as **exploration over a heterogeneous tool ecology**. The LLM generates pseudo-tool descriptions as retrieval probes—hypotheses about needed functionality—and refines them adaptively as the agent's task understanding evolves. We instantiate the framework as a spectrum of strategies trading speed for thoroughness: Single-Pass, Multi-Turn Refinement, and Scattershot.
- We introduce **Memetic Retrieval**, a multi-generation evolutionary loop over pseudo-tool descriptions with crossover, mutation, retrieval-fitness selection, and a tool memory that suppresses redundant search across generations. Memetic achieves the highest average pass rate on StableToolBench (0.73) and the best average retrieval rank on ToolRet (2.78), with the largest gains concentrated on ambiguous, multi-tool tasks (G2/G3) where deterministic retrieval is weakest.
- We find empirically that **retrieval, not planning, is the binding constraint** on end-to-end agent performance: scaling the analysis model improves retrieval across all strategies, while scaling the planner does not. This redirects attention from planning-centric agent improvements to retrieval-side capacity.
- We show that **memetic search is base-model contingent**: gains require an LLM capable of acting as a competent semantic operator on pseudo-tool descriptions. Under weaker base models, both diversity- and depth-generating moves collapse together—evolutionary search amplifies noise rather than refining signal, inverting AlphaEvolve's frontier-model result and bounding when memetic strategies should be deployed.

2 Problem Statement

We frame tool retrieval as a **zero-shot retrieval and decision-making problem**: given a task goal derived from user query q, the agent must identify, possibly across multiple steps, the most relevant tools from $\mathcal{T}$ without task-specific training data or hardcoded retrieval pipelines. This requires reasoning over both the query structure and the *affordances* of candidate tools, bridging the gap between semantic understanding of tasks and the operational use of tools in execution.

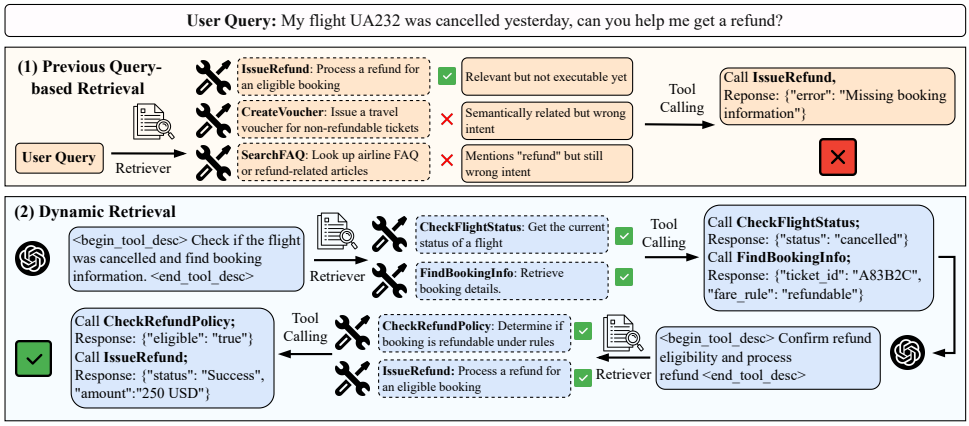

Figure 1: Comparison between Previous Query-based Retrieval and Dynamic Retrieval. The prior query-based retrieval fails to select the correct tool due to static dependency on earlier queries, while the proposed dynamic retrieval adaptively updates the retrieval context based on intermediate tool outputs, leading to successful task completion.

Formally, the agent generates a sequence of actions $a_1, a_2, \ldots, a_T$ where $a_t \in \mathcal{A}$. At each step t, the available actions depend on the *function schema* $\mathcal{F}_t \subseteq \mathcal{T}$, the subset of tools currently exposed to the agent from the full tool corpus $\mathcal{T}$. In long-horizon tasks, retrieval errors at early steps compound across the action sequence (Song et al., 2026), making the quality of $\mathcal{F}_t$ the binding constraint on task success.

Strategy trigger. The framework permits retrieval to be invoked during reasoning: at any step t, the agent may emit a retrieval request, causing the system to update the function schema $\mathcal{F}_{t+1} \leftarrow \mathcal{F}_t \cup \text{Retrieve}(a_t, \mathcal{T})$. The trigger mechanism and its implementation are detailed in §4.2.

Desiderata. We seek a tool retrieval framework satisfying the following properties: **(i) Model-agnostic:** the framework must operate with any base LLM without architecture-specific modifications; **(ii) Training-free:** it must not require fine-tuning the retriever or LLM, enabling deployment with API-only models; **(iii) Orthogonal:** the framework composes with advances in agentic planning and stronger base models without requiring co-adaptation, serving as a modular, plug-and-play capability.

3 Related Work

3.1 Tool-Augmented Large Language Models

Tool-augmented LLM agents enhance reasoning through external tool invocation. Early systems connected models to specific environments such as web browsers and APIs (Nakano et al., 2021; Song et al., 2023; Wang et al., 2024), while ReAct (Yao et al., 2022) established observation-action reasoning loops. Subsequent work splits into training-based methods that fine-tune on tool-call datasets (Schick et al., 2023; Patil et al., 2024) and training-free approaches that inject tool schemas into prompts for in-context selection, as exemplified and evaluated by MetaTool (Huang et al., 2023). However, these methods assume oracle tools are provided, which is unrealistic.

3.2 Tool Retrieval for LLM Agents

Tool retrieval aims to identify the top-K most relevant tools from a large repository for a given user query, serving as the foundation for effective tool use in LLM-based agents. While

tool-augmented LLM research has concentrated on tool *usage* and calling, with planning-centric approaches such as ToolChain* (Zhuang et al., 2023) improving action sequencing, the upstream retrieval bottleneck remains comparatively underexplored, with errors in tool selection propagating through inference trees (Chen et al., 2025). Recent work on dynamic tool discovery demonstrates that on-demand retrieval yields superior scalability for long-horizon tasks, even without additional training (Li et al., 2025c). Early approaches adopt conventional information retrieval techniques, which can be broadly divided into **term-based** and **semantic-based** methods. Term-based models such as TF–IDF (Sparck Jones, 1972) and BM25 (Robertson et al., 2009) rely on sparse lexical overlap between the query and tool documentation, often failing when users describe tool functionalities in natural or task-specific language.

To capture deeper semantic relations, recent research shifts toward semantic retrieval inspired by RAG (Lewis et al., 2020), encoding queries and tool descriptions into a shared dense embedding space. Representative works include Tool2Vec (Moon et al., 2024), which models diverse user invocation patterns to reduce the semantic gap between user requests and API specifications. Further refinements introduce hierarchical or specialized retrieval architectures: AnyTool (Du et al., 2024) and ToolRerank (Zheng et al., 2024) perform coarse-to-fine selection across category-tool-API levels, while COLT (Qu et al., 2024a) designs completeness-oriented reranking and Re-Invoke (Chen et al., 2024) applies invocation rewriting to improve precision. MCP-Zero (Fei et al., 2025) preliminarily explores allowing LLMs to actively request tools, but its approach focuses on conceptual formulation with minimal refinement or experimental validation. Concurrently, ToolDreamer (Sengupta et al., 2025) proposes instilling LLM reasoning into tool retrievers through training, and Xu et al. (2024) enhance retrieval with iterative LLM feedback, while our approach achieves similar goals at inference time without parameter updates. ToolGen (Wang et al., 2025) explores a generative approach to unified tool retrieval and calling, contrasting with our retrieval-augmented framework that decouples tool discovery from invocation. Despite these advances, existing retrieval-based frameworks face several fundamental limitations. **(1) Static retrieval:** most methods retrieve tools only once, preventing adaptation as reasoning evolves (§1). **(2) Task-specific or structure-dependent designs:** many methods rely on additional assumptions such as hierarchical tool taxonomies, pre-collected diverse user invocation patterns, or structured API documentation, which restricts their applicability to broader, unstructured tool environments where only user query and raw tool descriptions are available. **(3) Semantic gap:** direct query-to-tool matching yields poor recall (§1); prior work aligns tool embeddings toward queries, while we invert this direction via pseudo-tool generation.

Connection to prompt optimization. Automatic prompt optimization formulates search over discrete textual spaces (Li et al., 2025b), with recent work applying LLM-driven evolutionary operators (Agrawal et al., 2025) and strategic exploration–exploitation trade-offs (Cui et al., 2025). While these methods optimize instruction prompts scored by a downstream objective, our framework searches over pseudo-tool descriptions, textual artifacts whose fitness is evaluated by retrieval quality against the tool corpus rather than end-task performance.

4 Method

4.1 Overview

We frame tool retrieval as **tool exploration**: the API catalog constitutes a large, heterogeneous *tool ecology*, and the user query defines an ecological niche that the agent must fill by discovering well-adapted tools through imperfect retrieval signals. During reasoning, the agent searches this ecology by generating *pseudo-tool descriptions*, intermediate hypotheses about needed functionality that serve as retrieval probes (§4.2).

We instantiate this framework as a progression of strategies: *Single-Pass* generates one hypothesis; *Multi-Turn* iteratively refines hypothesis using retrieval feedback; *Scattershot* explores diverse hypotheses in parallel via voting; and *Memetic* subsumes both through

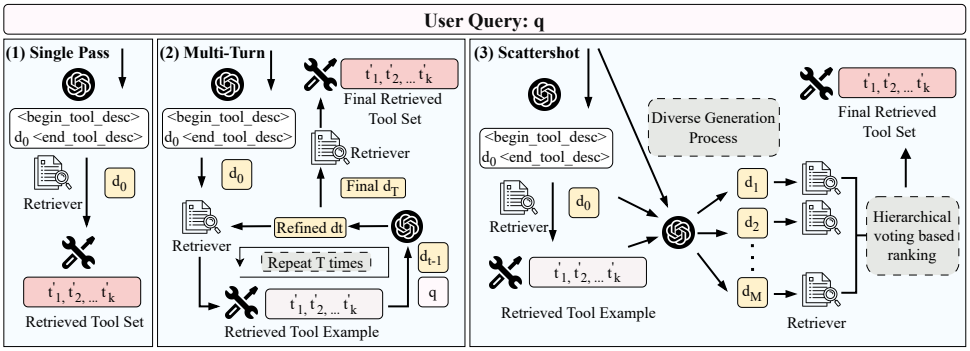

Figure 2: Illustration of our proposed dynamic tool retrieval framework. The figure compares the workflows of *Single-Pass*, *Multi-Turn Refinement*, and *Scattershot Retrieval*.

an evolutionary loop (Guo et al., 2025) with fitness-driven selection, crossover, mutation, and local search. Each strategy addresses a specific limitation of its predecessor, forming a spectrum from fast (Single-Pass) to thorough (Memetic).

4.2 Dynamic Tool Retrieval Framework

When a strategy trigger fires (§2), the system extracts one or more **pseudo-tool descriptions** $d = \text{LLM}(q, \mathcal{H})$ from the agent's output a_t. Concretely, the output contains pseudo-tool blocks delimited by special tokens ($\{\texttt{BEGIN}\} \ldots \{\texttt{END}\}$); upon detection, the system intercepts a_t, extracts each block, and runs the configured retrieval strategy. Because generating a tool hypothesis requires reasoning about expected inputs and outputs, each pseudo-tool implicitly encodes a more granular task plan than explicit planning approaches such as cluster-based tool selection (Liu et al., 2025).

Retrieval mechanism. Given a pseudo-tool description d and the tool corpus $\mathcal{T} = \{t_1, \ldots, t_n\}$, we compute relevance via embedding similarity. Let $\mathbf{e}(\cdot)$ denote a sentence embedding function (SimCSE (Gao et al., 2021), RoBERTa-large variant). We compute the relevance score as:

$$\text{score}(d, t_i) = \cos\big(\mathbf{e}(d),\, \mathbf{e}(t_i)\big), \tag{1}$$

and retrieve the top-k tools $R = \textsc{Ret}(d, \mathcal{T}, k)$ by descending score. Retrieved tools are injected into the function schema $\mathcal{F}_{t+1} \leftarrow \mathcal{F}_t \cup R$, enabling the agent to invoke newly discovered tools in subsequent steps.

Single-Pass Retrieval. In the simplest instantiation, the initial description $d_0 = \text{LLM}(q, \mathcal{H})$ is used directly for a single retrieval step with no further refinement. This establishes the benefit of LLM-generated pseudo-tools over raw query retrieval: even a single pseudo-tool description bridges the semantic gap by operating in a space closer to tool documentation than the original user query.

However, Single-Pass provides no mechanism to correct misalignment between d_0 and the tool database after the initial retrieval.

4.3 Multi-Turn Refinement

The initial pseudo-tool description d_0 may not align well with actual tool representations in $\mathcal{T}$. Drawing on the iterative self-refinement paradigm (Madaan et al., 2023), we propose a multi-turn refinement mechanism that uses retrieval feedback to progressively improve description quality.

Intuition. After each retrieval step, the agent receives concrete examples of tools that *actually exist* in the database. These exemplars provide a grounding signal by revealing

the vocabulary, structure, and granularity of real tool descriptions that the LLM can use to refine its hypothesis.

Formal description. Let $d^{(0)} = d_0$ be the initial pseudo-tool description. At each refinement turn $t = 1, \ldots, T$:

$$R^{(t)} = \textsc{Ret}\big(d^{(t-1)},\, \mathcal{T},\, k\big), \tag{2}$$

$$\mathcal{B}^{(t)} = \mathcal{B}^{(t-1)} \cup \big\{\text{blurb}(r) : r \in R^{(t)}\big\}, \tag{3}$$

$$d^{(t)} = \text{LLM}_{\text{refine}}\big(d^{(t-1)},\, \mathcal{B}^{(t)},\, q\big), \tag{4}$$

where $\mathcal{B}^{(t)}$ is the accumulated set of exemplar tool descriptions (deduplicated) and $\text{blurb}(r)$ extracts a normalized natural-language summary of tool r. The refinement operator $\text{LLM}_{\text{refine}}$ generates an updated pseudo-tool description conditioned on (i) the current description, (ii) the exemplar set, and (iii) the original query. The final retrieval uses $d^{(T)}$:

$$\text{Output} = \textsc{Ret}\big(d^{(T)},\, \mathcal{T},\, k\big). \tag{5}$$

The temperature parameter τ is set low ($\tau \leq 0.7$) to encourage focused, incremental refinement rather than divergent exploration.

While effective, Multi-Turn follows a single deterministic trajectory that may converge to a local optimum when d_0 is poorly formulated.

4.4 Scattershot Retrieval

To overcome the single-trajectory limitation of Multi-Turn, Scattershot Retrieval introduces population-level diversity by generating multiple diverse pseudo-tool descriptions and aggregating their retrieval results through voting.

Intuition. A single pseudo-tool description may miss relevant tools due to the inherent ambiguity of natural-language generation. By sampling M diverse descriptions at high temperature, the framework explores a broader region of the pseudo-tool space, increasing the probability that at least one description aligns well with each required tool.

Diverse generation. For each initial description d_0, we first perform a seed retrieval $R_0 = \textsc{Ret}(d_0, \mathcal{T}, k)$ and extract exemplars $\mathcal{E} = \{\text{blurb}(r) : r \in R_0\}$. We then generate M diverse children in parallel at elevated temperature $\tau = 1.5$:

$$d_i \;\sim\; \text{LLM}_{\text{scatter}}\big(d_0,\, \mathcal{E};\, \tau\big), \quad i = 1, \ldots, M. \tag{6}$$

Hierarchical voting. Each child d_i independently retrieves its own candidate set $R_i = \textsc{Ret}(d_i, \mathcal{T}, k)$. Results are aggregated by vote count, mean rank, and mean similarity (Algorithm 7), ensuring tools consistently retrieved across diverse hypotheses are prioritized.

4.5 Memetic Retrieval

Scattershot generates a diverse population but discards all information after a single generation. We now present **Memetic Retrieval** algorithm, which combines population-based exploration with iterative refinement through a multi-generation evolutionary loop.

Intuition. Each pseudo-tool description acts as a mutable hypothesis, analogous to a *meme* in cultural evolution, whose quality is evaluated by retrieval outcomes. The algorithm maintains a *population* of competing hypotheses, applies *selection pressure* via a fitness signal, and generates new candidates through *crossover* and *mutation*. The LLM serves as a *semantic* evolutionary operator (Guo et al., 2025; Agrawal et al., 2025), performing meaning-preserving crossover and intent-aware mutation at the meaning level rather than token-level perturbation (Novikov et al., 2025). A memetic local search step refines each offspring using retrieval feedback.

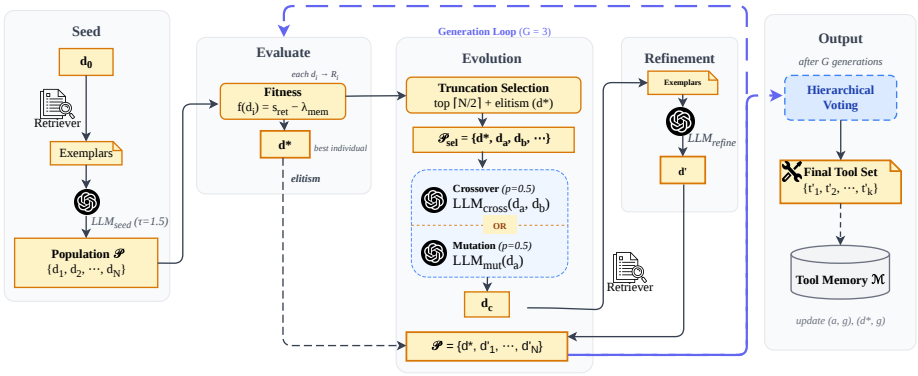

Figure 3: Overview of Memetic Retrieval. The framework progresses through five phases: **Seed** generates an initial population of pseudo-tool descriptions; **Evaluate** scores each via retrieval fitness; **Evolution** applies truncation selection with crossover and mutation; **Refinement** refines offspring through retrieval feedback; and **Output** aggregates results via hierarchical voting. The generation loop repeats for G iterations.

Fitness function. The fitness of a pseudo-tool description d_i with retrieval result $R_i = \textsc{Ret}(d_i, \mathcal{T}, k)$ is:

$$f(d_i) = \underbrace{s_{\text{ret}}(d_i, R_i)}_{\text{retrieval quality}} \underbrace{- 0.5 \cdot \max_{(M_j, g_j) \in \mathcal{M}} J(R_i, M_j) \cdot \mathbb{1}\big[J(R_i, M_j) > 0.3\big]}_{\text{memory penalty}}, \tag{7}$$

where $s_{\text{ret}}(d_i, R_i) = 0.7\,\sigma_1(R_i) + 0.3\,\bar{\sigma}_3(R_i)$ combines top-1 and mean top-3 cosine similarities, $J(\cdot,\cdot)$ is Jaccard similarity, and $\mathcal{M} = \{(M_j, g_j)\}$ is the tool memory storing previously retrieved tool sets with generation indices. The retrieval score operates at the *semantic* level (embedding similarity), while the Jaccard penalty operates at the *syntactic* level (set overlap); together they discourage redundant exploration. Because API outcomes are stochastic, this memory acts as a soft taboo rather than a hard ban, recording what *not* to revisit (Suzgun et al., 2025). At the population level, the penalty implicitly coordinates exploration by steering candidates away from already-covered regions.

5 Experiments

5.1 Experimental Setup

We evaluate on two complementary benchmarks: **ToolRet** (Shi et al., 2025) (43k tools) for retrieval-only evaluation at scale, and **StableToolBench** (Guo et al., 2024) (16,464 APIs), a stabilized variant of ToolBench (Qin et al., 2024b) for end-to-end tool use. Without oracle tool sets, retrieval quality binds end-to-end performance on both benchmarks (Shi et al., 2025). We compare **Query Retrieval** (static baseline), **Single-Pass** (§4.2), **Multi-Turn Refinement** (§4.3), **Scattershot Retrieval** (§4.4), and **Memetic** (§4.5) using *GPT-4.1-mini* and *Qwen3-30B* as base models. We report *NDCG@5*, *Precision@5*, *Recall@5*, and *Comprehensiveness@5*. StableToolBench organizes tasks into three complexity levels (G1/G2/G3) based on the number of tools required; see Appendix A.3 for definitions. Full setup details (tool corpus construction, base model configurations, and metric definitions) appear in Appendix A.2.

5.2 Experiment Results on Retrieval-Only Task.

Table 1 summarizes retrieval-only performance on the *ToolRet* benchmark across four domains (*Code*, *Customized*, *Web*, and *ToolBench*). We observe:

Algorithm 1 Memetic Retrieval

Require: $\mathcal{D}_0$, $\mathcal{T}$, k, $\mathcal{M}$, N=6, G=3, θ=0.95, τ=1.5, B
Ensure: Ranked tool list; updated $\mathcal{M}$

for each ancestor $a \in \mathcal{D}_0$ **do**
  $\mathcal{P} \leftarrow \textsc{SeedPop}(a, \mathcal{T}, k, N, \tau)$ *Alg. 6*
  for $g \leftarrow 1$ to G **do**
    for each $d_i \in \mathcal{P}$ **do** $R_i \leftarrow \textsc{Ret}(d_i, \mathcal{T}, k)$; $f_i \leftarrow f(d_i)$ *Eq. 7*
    end for
    $d^* \leftarrow \arg\max_i f_i$; **if** $f(d^*) \geq \theta$ or $g = G$ **then break**
    $\mathcal{P}_{\text{sel}} \leftarrow \text{top}_{\lceil N/2 \rceil}(\mathcal{P}, f)$; $\mathcal{P}' \leftarrow \{d^*\}$ *elitism*
    while $|\mathcal{P}'| < N$ **do** *crossover or mutation*
      if $\text{Uniform}(0,1) < 0.5$ **then** $d_a, d_b \leftarrow \text{sample}(\mathcal{P}_{\text{sel}}, 2)$; $d_c \leftarrow \text{LLM}_{\text{cross}}(d_a, d_b, a;\ \tau)$
      else $d_a \leftarrow \text{sample}(\mathcal{P}_{\text{sel}}, 1)$; $d_c \leftarrow \text{LLM}_{\text{mut}}(d_a, a;\ \tau)$
      end if
      $\mathcal{P}' \leftarrow \mathcal{P}' \cup \{d_c\}$
    end while
    for each $d \in \mathcal{P}' \setminus \{d^*\}$ **do** $d \leftarrow \text{LLM}_{\text{refine}}(a, d, \textsc{Ret}, q)$ *local search*
    end for
    $\mathcal{P} \leftarrow \mathcal{P}'$
  end for
  $\mathcal{M}$ += $\{(a, g), (d^*, g)\}$; *keys*.extend($\textsc{Vote}(\mathcal{P}, \mathcal{T}, k, B)$) *Alg. 7*
end for
return *keys*

Table 1: Main results on retrieval-only setting. Each dataset reports four metrics: NDCG@5 (N@5), Precision@5 (P@5), Recall@5 (R@5), and Comprehensiveness@5 (C@5). Bold indicates best within each base model group; underlined bold indicates best across both groups.

Base Model	Strategy	Code				Customized				Web				ToolBench				Avg. Rank
		N@5	P@5	R@5	C@5	N@5	P@5	R@5	C@5	N@5	P@5	R@5	C@5	N@5	P@5	R@5	C@5	
—	Query Retrieval	27.93	8.95	33.33	30.53	30.59	13.22	34.18	23.32	29.92	10.52	35.63	26.81	22.88	11.78	27.27	8.72	8.81
GPT-4.1-mini	Single-Pass	31.49	9.79	37.89	34.48	34.88	15.48	37.05	24.44	32.33	11.94	38.88	29.52	24.25	13.07	29.99	11.55	6.41
	Multi-Turn	34.96	10.81	41.22	37.39	34.58	15.58	37.13	24.34	33.79	**12.53**	**39.72**	**30.12**	30.98	15.27	34.49	15.55	3.91
	Scattershot	**35.43**	10.82	**41.77**	**38.19**	35.25	15.68	**38.40**	**25.56**	32.54	12.03	38.54	28.72	27.41	14.20	32.38	12.73	4.25
	Memetic	35.18	**11.04**	40.76	36.99	**35.75**	**16.31**	37.95	24.34	**33.84**	12.51	39.33	29.94	**32.05**	**15.71**	**35.48**	**17.09**	**2.78**
Qwen3-30B	Single-Pass	31.48	9.86	37.92	34.59	**35.10**	**15.58**	**37.81**	**24.85**	31.98	12.14	**38.82**	29.16	25.08	13.47	30.66	11.55	5.62
	Multi-Turn	**37.64**	**11.30**	43.33	39.34	34.17	**15.58**	36.34	23.12	**32.64**	**12.36**	38.63	29.18	**34.43**	**16.96**	**37.83**	**18.18**	**3.44**
	Scattershot	37.17	11.23	**44.15**	**40.25**	34.72	15.54	37.38	24.64	32.61	12.12	38.66	**29.37**	29.43	33.04	14.64	14.91	3.94
	Memetic	34.96	10.98	41.14	37.11	33.03	**15.58**	36.05	23.12	31.49	11.78	37.43	28.64	31.51	15.53	35.12	16.45	5.84

Table 2: End-to-end pass rates on StableToolBench (16,464 APIs). Avg is the mean across all 6 scenarios. All dynamic methods use GPT-4.1-mini. Per-variant breakdowns and Qwen3-30B results appear in Tables 3 and 4.

	Pass Rate						
Strategy	**G1-I**	**G1-T**	**G1-C**	**G2-I**	**G2-C**	**G3-I**	**Avg**
Query Retrieval	0.56	0.56	0.53	0.42	0.48	0.42	0.49
Single-Pass	0.59	0.60	0.61	0.55	0.57	0.51	0.57
Multi-Turn	0.61	0.59	0.69	0.51	0.51	0.48	0.56
Scattershot	0.69	0.61	0.63	0.56	0.55	0.53	0.59
Memetic	**0.72**	**0.69**	**0.78**	**0.70**	**0.76**	**0.75**	**0.73**

- **LLM-guided retrieval outperforms query retrieval.** All three LLM-guided strategies outperform *Query Retrieval* (Avg. Rank 8.81) across every dataset and metric, confirming that pseudo-tool descriptions bridge the query–tool semantic gap.
- **Refinement and diversity improve retrieval.** Both *Multi-Turn* and *Scattershot* achieve strong average ranks (3.44–3.91 and 3.94–4.25, respectively), yielding notable gains over *Single-Pass* (5.62–6.41) and confirming that iterative feedback and diverse formulations both enhance retrieval.

- **Complementary strengths.** *Scattershot* excels on *Code* and *Customized* (broader phrasing variation favors diverse descriptions), while *Multi-Turn* dominates *Web* and *ToolBench* (higher tool similarity rewards iterative refinement).
- **Model consistency.** The relative ordering among strategies remains stable across both *GPT-4.1-mini* and *Qwen3-30B*, with *Memetic* achieving the best average rank under *GPT-4.1-mini* (2.78) and *Multi-Turn* leading under *Qwen3-30B* (3.44).
- **Memetic is competitive but not dominant in retrieval-only.** With *GPT-4.1-mini*, *Memetic* leads on *Customized* (N@5 35.75, P@5 16.31) and *Web* (N@5 33.84) but trails *Scattershot* on *Code*. With *Qwen3-30B*, *Memetic* underperforms all other strategies, consistent with the end-to-end degradation observed in Table 4. *Memetic*'s gains concentrate in the end-to-end setting (Table 2), where tool-execution feedback supplies a signal the retrieval-only benchmark does not.

5.3 Experiment Results on End-to-end Tool Use Task

Table 2 reports pass rates on StableToolBench. The trends mirror the retrieval-only results: all LLM-guided strategies outperform *Query Retrieval* (0.49 avg), with *Scattershot* (0.59) and *Single-Pass* (0.57) both surpassing *Multi-Turn* (0.56) in this end-to-end setting. *Memetic* achieves the highest average pass rate (0.73), a **24-point absolute gain** over static *Query Retrieval* (0.49) and a 14-point gain over the next-best dynamic method. The improvement is most pronounced on the multi-tool G2 and G3 subsets (Appendix A.3): Memetic reaches 0.70–0.76 versus 0.42–0.48 for Query Retrieval, a 22–34-point absolute gain on the regimes where queries are ambiguous and lexical cues are weak. Memetic turns that ambiguity into an exploration signal by searching over alternative pseudo-tool descriptions rather than committing to a single rule-based match. Full per-variant pass rates and Qwen3-30B results appear in Tables 3 and 4 (Appendix A.4).

5.4 Ablation Study

We ablate refinement depth and model choice (Appendix A.1).

Single-trajectory refinement plateaus. Scaling Multi-Turn beyond $T=1$ yields diminishing returns: because refinement follows a single trajectory, it converges to local optima rather than exploring the description space broadly. This motivates population-based strategies (Scattershot, Memetic) that maintain diverse candidate pools instead of deepening a single refinement chain.

Retrieval, not planning, is the bottleneck. A stronger analysis model improves retrieval across all strategies, whereas a stronger planner does not (Sinha et al., 2026). The binding constraint on end-to-end performance is retrieval quality, not downstream reasoning—scaling the retriever-side LLM produces gains that scaling the planner does not.

Memetic exploration inverts under weak base models. With a weaker base model (Qwen3-30B), Memetic collapses from 73.4% pass rate (under GPT-4.1-mini) to 32.2%—below even static Query Retrieval (34.8%). *Both* the diversity-generating moves (crossover/mutation, larger populations) and the depth-generating moves (iterative refinement) degrade together: deeper refinement and larger populations actively reduce pass rates rather than improving them (Appendix A.1). This inverts AlphaEvolve's frontier-model finding (Novikov et al., 2025): where a strong LLM compounds diversity into discovery, a weaker one compounds it into noise. Memetic exploration therefore presupposes an LLM capable of acting as a competent semantic operator on pseudo-tool descriptions; absent that, evolutionary search amplifies generation-over-generation noise rather than refining offspring.

Although Memetic spends more tokens during pseudo-tool generation, that cost is paid once at retrieval time and amortized across the episode because discovered tools persist in the active function schema; the marginal cost is small relative to downstream reasoning and execution.

6 Conclusion

We introduced `FitText`, a framework that embeds tool retrieval directly into the agent's reasoning loop, using LLM-generated pseudo-tool descriptions to bridge the semantic gap between user intent and tool metadata. By combining iterative refinement, stochastic diversity, and evolutionary search with persistent tool memory, `FitText` adaptively narrows large tool spaces without any task-specific training. Our results demonstrate consistent and substantial gains over static retrieve-then-act pipelines, with the largest improvements on ambiguous, multi-tool tasks. These findings establish adaptive, in-loop retrieval as a general-purpose interface for evolving tool ecosystems. Memetic search is not free: it presupposes a base LLM strong enough to act as a competent semantic operator, and absent this capacity, evolutionary exploration amplifies noise rather than refining signal.

References

Lakshya A. Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziems, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J. Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alexandros G. Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. Gepa: Reflective prompt evolution can outperform reinforcement learning. *arXiv preprint arXiv:2507.19457*, 2025.

Akari Asai, Timo Schick, Patrick Lewis, Xilun Chen, Gautier Izacard, Sebastian Riedel, Hannaneh Hajishirzi, and Wen-tau Yih. Task-aware retrieval with instructions. *arXiv preprint arXiv:2211.09260*, 2023.

Sijia Chen, Yibo Wang, Yi-Feng Wu, Qing-Guo Chen, Zhao Xu, Weihua Luo, Kaifu Zhang, and Lijun Zhang. Advancing tool-augmented large language models: Integrating insights from errors in inference trees. *arXiv preprint arXiv:2406.07115*, 2025.

Yanfei Chen, Jinsung Yoon, Devendra Sachan, Qingze Wang, Vincent Cohen-Addad, Mohammadhossein Bateni, Chen-Yu Lee, and Tomas Pfister. Re-invoke: Tool invocation rewriting for zero-shot tool retrieval. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pp. 4705–4726, 2024.

Alejandro Cuadron, Dacheng Li, Wenjie Ma, Xingyao Wang, Yichuan Wang, Siyuan Zhuang, Shu Liu, Luis Gaspar Schroeder, Tian Xia, Huanzhi Mao, Nicholas Thumiger, Aditya Desai, Ion Stoica, Ana Klimovic, Graham Neubig, and Joseph E. Gonzalez. The danger of overthinking: Examining the reasoning-action dilemma in agentic tasks. *arXiv preprint arXiv:2502.08235*, 2025.

Wendi Cui, Zhuohang Li, Hao Sun, Damien Lopez, Kamalika Das, Bradley Malin, Sricharan Kumar, and Jiaxin Zhang. See: Strategic exploration and exploitation for cohesive in-context prompt optimization. *arXiv preprint arXiv:2402.11347*, 2025.

Yu Du, Fangyun Wei, and Hongyang Zhang. Anytool: Self-reflective, hierarchical agents for large-scale api calls. *arXiv preprint arXiv:2402.04253*, 2024.

Xiang Fei, Xiawu Zheng, and Hao Feng. Mcp-zero: Proactive toolchain construction for llm agents from scratch. *arXiv preprint arXiv:2506.01056*, 2025.

Luyu Gao, Xueguang Ma, Jimmy Lin, and Jamie Callan. Precise zero-shot dense retrieval without relevance labels. *arXiv preprint arXiv:2212.10496*, 2022.

Tianyu Gao, Xingcheng Yao, and Danqi Chen. Simcse: Simple contrastive learning of sentence embeddings. In *2021 Conference on Empirical Methods in Natural Language Processing, EMNLP 2021*, pp. 6894–6910. Association for Computational Linguistics (ACL), 2021.

Qingyan Guo, Rui Wang, Junliang Guo, Bei Li, Kaitao Song, Xu Tan, Guoqing Liu, Jiang Bian, and Yujiu Yang. Evoprompt: Connecting llms with evolutionary algorithms yields powerful prompt optimizers. *arXiv preprint arXiv:2309.08532*, 2025.

Zhicheng Guo, Sijie Cheng, Hao Wang, Shihao Liang, Yujia Qin, Peng Li, Zhiyuan Liu, Maosong Sun, and Yang Liu. Stabletoolbench: Towards stable large-scale benchmarking on tool learning of large language models. In *ACL (Findings)*, 2024.

Xu Huang, Weiwen Liu, Xiaolong Chen, Xingmei Wang, Hao Wang, Defu Lian, Yasheng Wang, Ruiming Tang, and Enhong Chen. Understanding the planning of llm agents: A survey. *arXiv preprint arXiv:2402.02716*, 2024.

Yue Huang, Jiawen Shi, Yuan Li, Chenrui Fan, Siyuan Wu, Qihui Zhang, Yixin Liu, Pan Zhou, Yao Wan, Neil Zhenqiang Gong, et al. Metatool benchmark for large language models: Deciding whether to use tools and which to use. *arXiv preprint arXiv:2310.03128*, 2023.

Victor Lavrenko and W. Bruce Croft. Relevance based language models. *Proceedings of the 24th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, pp. 120–127, 2001.

Yibin Lei, Yu Cao, Tianyi Zhou, Tao Shen, and Andrew Yates. Corpus-steered query expansion with large language models. *arXiv preprint arXiv:2402.18031*, 2024.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, 33:9459–9474, 2020.

Hang Li, Kaiqi Yang, Yucheng Chu, Hui Liu, and Jiliang Tang. Exploring solution divergence and its effect on large language model problem solving. *arXiv preprint arXiv:2509.22480*, 2025a.

Wenwu Li, Xiangfeng Wang, Wenhao Li, and Bo Jin. A survey of automatic prompt engineering: An optimization perspective. *arXiv preprint arXiv:2502.11560*, 2025b.

Xiaoxi Li, Wenxiang Jiao, Jiarui Jin, Guanting Dong, Jiajie Jin, Yinuo Wang, Hao Wang, Yutao Zhu, Ji-Rong Wen, Yuan Lu, and Zhicheng Dou. Deepagent: A general reasoning agent with scalable toolsets. *arXiv preprint arXiv:2510.21618*, 2025c.

Zhicong Li, Jiahao Wang, Zhishu Jiang, Hangyu Mao, Zhongxia Chen, Jiazhen Du, Yuanxing Zhang, Fuzheng Zhang, Di Zhang, and Yong Liu. Dmqr-rag: Diverse multi-query rewriting for rag. *arXiv preprint arXiv:2411.13154*, 2024.

Jie Liu and Barzan Mozafari. Query rewriting via large language models. *arXiv preprint arXiv:2403.09060*, 2024.

Yanming Liu, Xinyue Peng, Jiannan Cao, Shi Bo, Yuwei Zhang, Xuhong Zhang, Sheng Cheng, Xun Wang, Jianwei Yin, and Tianyu Du. Tool-planner: Task planning with clusters across multiple tools. *arXiv preprint arXiv:2406.03807*, 2025.

Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback. *arXiv preprint arXiv:2303.17651*, 2023.

Shervin Minaee, Tomas Mikolov, Narjes Nikzad, Meysam Chenaghlu, Richard Socher, Xavier Amatriain, and Jianfeng Gao. Large language models: A survey. *arXiv preprint arXiv:2402.06196*, 2024.

Suhong Moon, Siddharth Jha, Lutfi Eren Erdogan, Sehoon Kim, Woosang Lim, Kurt Keutzer, and Amir Gholami. Efficient and scalable estimation of tool representations in vector space. *arXiv preprint arXiv:2409.02141*, 2024.

Reiichiro Nakano, Jacob Hilton, Suchir Balaji, Jeff Wu, Long Ouyang, Christina Kim, Christopher Hesse, Shantanu Jain, Vineet Kosaraju, William Saunders, et al. Webgpt: Browser-assisted question-answering with human feedback. *arXiv preprint arXiv:2112.09332*, 2021.

Alexander Novikov, Ngân Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, and Matej Balog. Alphaevolve: A coding agent for scientific and algorithmic discovery. *arXiv preprint arXiv:2506.13131*, 2025.

Bhargavi Paranjape, Scott Lundberg, Sameer Singh, Hannaneh Hajishirzi, Luke Zettlemoyer, and Marco Tulio Ribeiro. Art: Automatic multi-step reasoning and tool-use for large language models. *arXiv preprint arXiv:2303.09014*, 2023.

Shishir G Patil, Tianjun Zhang, Xin Wang, and Joseph E Gonzalez. Gorilla: Large language model connected with massive apis. *Advances in Neural Information Processing Systems*, 37: 126544–126565, 2024.

Yujia Qin, Shengding Hu, Yankai Lin, Weize Chen, Ning Ding, Ganqu Cui, Zheni Zeng, Xuanhe Zhou, Yufei Huang, Chaojun Xiao, et al. Tool learning with foundation models. *ACM Computing Surveys*, 57(4):1–40, 2024a.

Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, et al. Toolllm: Facilitating large language models to master 16000+ real-world apis. In *The Twelfth International Conference on Learning Representations*, 2024b.

Changle Qu, Sunhao Dai, Xiaochi Wei, Hengyi Cai, Shuaiqiang Wang, Dawei Yin, Jun Xu, and Ji-Rong Wen. Colt: Towards completeness-oriented tool retrieval for large language models. *arXiv e-prints*, pp. arXiv–2405, 2024a.

Changle Qu, Sunhao Dai, Xiaochi Wei, Hengyi Cai, Shuaiqiang Wang, Dawei Yin, Jun Xu, and Ji-Rong Wen. Towards completeness-oriented tool retrieval for large language models. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management*, pp. 1930–1940, 2024b.

Changle Qu, Sunhao Dai, Xiaochi Wei, Hengyi Cai, Shuaiqiang Wang, Dawei Yin, Jun Xu, and Ji-Rong Wen. Tool learning with large language models: A survey. *Frontiers of Computer Science*, 19(8):198343, 2025.

Stephen Robertson, Hugo Zaragoza, et al. The probabilistic relevance framework: Bm25 and beyond. *Foundations and Trends® in Information Retrieval*, 3(4):333–389, 2009.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems*, 36: 68539–68551, 2023.

Saptarshi Sengupta, Zhengyu Zhou, Jun Araki, Xingbo Wang, Bingqing Wang, Suhang Wang, and Zhe Feng. Tooldreamer: Instilling llm reasoning into tool retrievers. *arXiv preprint arXiv:2510.19791*, 2025.

Yongliang Shen, Kaitao Song, Xu Tan, Dongsheng Li, Weiming Lu, and Yueting Zhuang. Hugginggpt: Solving ai tasks with chatgpt and its friends in hugging face. *Advances in Neural Information Processing Systems*, 36:38154–38180, 2023.

Zhengliang Shi, Yuhan Wang, Lingyong Yan, Pengjie Ren, Shuaiqiang Wang, Dawei Yin, and Zhaochun Ren. Retrieval models aren't tool-savvy: Benchmarking tool retrieval for large language models. *arXiv preprint arXiv:2503.01763*, 2025.

Akshit Sinha, Arvindh Arun, Shashwat Goel, Steffen Staab, and Jonas Geiping. The illusion of diminishing returns: Measuring long horizon execution in llms. *arXiv preprint arXiv:2509.09677*, 2026.

Peiyang Song, Pengrui Han, and Noah Goodman. Large language model reasoning failures. *arXiv preprint*, 2026.

Yifan Song, Weimin Xiong, Dawei Zhu, Wenhao Wu, Han Qian, Mingbo Song, Hailiang Huang, Cheng Li, Ke Wang, Rong Yao, et al. Restgpt: Connecting large language models with real-world restful apis. *arXiv preprint arXiv:2306.06624*, 2023.

Karen Sparck Jones. A statistical interpretation of term specificity and its application in retrieval. *Journal of documentation*, 28(1):11–21, 1972.

Mirac Suzgun, Mert Yuksekgonul, Federico Bianchi, Dan Jurafsky, and James Zou. Dynamic cheatsheet: Test-time learning with adaptive memory. *arXiv preprint arXiv:2504.07952*, 2025.

Liang Wang, Nan Yang, and Furu Wei. Query2doc: Query expansion with large language models. *arXiv preprint arXiv:2303.07678*, 2023.

Renxi Wang, Xudong Han, Lei Ji, Shu Wang, Timothy Baldwin, and Haonan Li. Toolgen: Unified tool retrieval and calling via generation. *arXiv preprint arXiv:2410.03439*, 2025.

Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. Executable code actions elicit better llm agents. In *Forty-first International Conference on Machine Learning*, 2024.

Qiancheng Xu, Yongqi Li, Heming Xia, and Wenjie Li. Enhancing tool retrieval with iterative feedback from large language models. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pp. 9609–9619, 2024.

Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V. Le, Denny Zhou, and Xinyun Chen. Large language models as optimizers. *arXiv preprint arXiv:2309.03409*, 2024.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *The eleventh international conference on learning representations*, 2022.

Yuanhang Zheng, Peng Li, Wei Liu, Yang Liu, Jian Luan, and Bin Wang. Toolrerank: Adaptive and hierarchy-aware reranking for tool retrieval. In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*, pp. 16263–16273, 2024.

Yuchen Zhuang, Xiang Chen, Tong Yu, Saayan Mitra, Victor Bursztyn, Ryan A. Rossi, Somdeb Sarkhel, and Chao Zhang. Toolchain*: Efficient action space navigation in large language models with a* search. *arXiv preprint arXiv:2310.13227*, 2023.

A Experimental Details

A.1 Ablation Studies

Effect of Query Analysis Model. We study how the model used for initial query analysis affects retrieval performance. As shown in Figure 4, using a stronger model (GPT-4.1) to interpret the user query leads to consistently higher retrieval scores across all datasets and strategies. This result suggests that our framework can naturally benefit from better query understanding, as the quality of the initial tool description directly influences subsequent refinement and retrieval. This trend implies that improving the initial task understanding allows even smaller models in later refinement steps to achieve strong performance.

Effect of Number of Refinement Turns. As shown in Figure 5, across most datasets, the improvement from `T0` to `T1` is the most significant, while later refinements bring only marginal gains. This trend aligns with intuition: the first refinement round is when the LLM first receives feedback from the tool database, allowing it to adjust its understanding of which tools actually exist and to optimize its generated descriptions accordingly. Subsequent iterations mainly provide fine-tuning rather than substantial corrections. An exception appears in the *Customized* dataset, where performance remains almost unchanged across

turns. This suggests that feedback from retrieved tools may be less informative in this domain, consistent with the observation that *Scattershot* performs better than *Multi-Turn* and that even *Single-Pass* already achieves strong results.

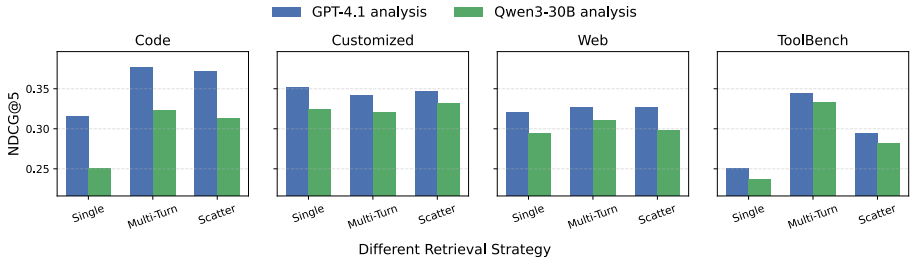

Figure 4: Effect of query analysis model on retrieval performance. Using GPT-4.1 as the analysis model consistently outperforms Qwen3-30B across all datasets and strategies, confirming that pseudo-tool quality benefits directly from stronger query understanding.

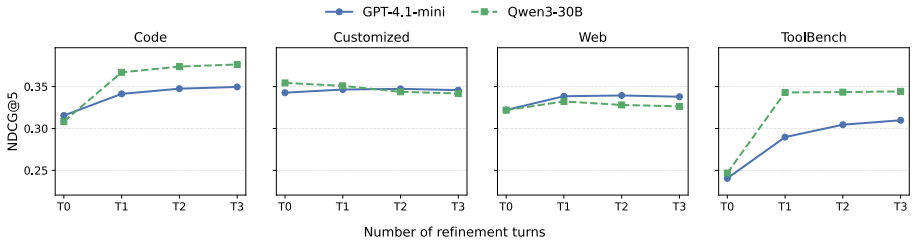

Figure 5: Effect of refinement turns on retrieval performance. The largest improvement occurs from T0 to T1; subsequent turns yield diminishing returns, suggesting that the first retrieval feedback round provides the most informative grounding signal.

A.2 Experimental Setup Details

Tool Corpus Construction. The original tool corpus contains detailed metadata for each tool, including parameters, request/response examples, and code snippets. Since our goal is to perform *semantic matching* based on tool descriptions, we simplify these representations by prompting *GPT-4.1-mini* to generate concise natural-language summaries for each tool. Given the full documentation as input, the model produces a short description capturing the tool's core functionality and purpose, removing low-level implementation details while preserving semantic meaning for retrieval.

Tasks and settings. We consider two evaluation tasks: (i) *retrieval-only*: given a single user query, the model retrieves the most relevant tools directly based on the query text without executing or calling any tools; and (ii) *end-to-end tool use*: the agent performs multi-turn reasoning in an executable environment, actively calling tools and consuming their responses to produce a final answer.

Base models. For retrieval-only tasks, we use two representative base models: the closed-source *GPT-4.1-mini* and the open-source *Qwen3-30B*. Since Qwen3-30B sometimes underperforms in complex query analysis, consistent with evidence that smaller models struggle with environmental comprehension in agentic settings, relying more heavily on internal reasoning chains (Cuadron et al., 2025), we let GPT-4.1 first reformulate the input query, while subsequent refinement (e.g., multi-turn or scattershot strategies) is performed by Qwen3-30B. For end-to-end tool-use tasks, we adopt a similar hybrid design: *o3-mini* assists in initial query understanding, and *GPT-4.1-mini* handles downstream reasoning, retrieval,

and execution. Notably, substituting a stronger planner did not improve pass rates in our experiments (§5), confirming that the bottleneck lies in retrieval quality rather than planning (Sinha et al., 2026).

Metrics. We evaluate retrieval performance using multiple standard IR metrics. *NDCG@5* measures ranking precision with position-aware weighting. *Comprehensiveness@5* assesses whether all ground-truth tools are successfully retrieved within the top results, reflecting holistic retrieval completeness. *Precision@5* and *Recall@5* further capture the accuracy and coverage of the retrieved tool set.

A.3 Benchmark Task Structure

Our end-to-end evaluation uses StableToolBench (Guo et al., 2024), a stabilized variant of ToolBench (Qin et al., 2024b) that replaces unstable real APIs with a virtual API server combining cached responses and LLM-based API simulators. The benchmark organizes tasks into three complexity levels based on how many tools are required, each further divided by how the test queries were sampled:

- **G1 (Single-tool):** Tasks solvable with a single API tool. The agent must identify and call one correct tool from the catalog. Three sampling sub-groups:
  - *G1-Inst (Instruction)*: Queries derived from human-written task instructions.
  - *G1-Cat (Category)*: Queries sampled at the tool category level.
  - *G1-Tool*: Queries sampled at the individual tool level.
- **G2 (Intra-category multi-tool):** Tasks requiring multiple tools from the *same* RapidAPI category. The agent must coordinate calls to related tools (e.g., two weather APIs from the "Weather" category). Sub-groups: *G2-Cat*, *G2-Inst*.
- **G3 (Intra-collection multi-tool):** Tasks requiring tools from *different* categories within the same RapidAPI collection. This is the hardest setting: the agent must discover and compose tools across category boundaries. Sub-group: *G3-Inst*.

The progression from G1 to G3 reflects increasing retrieval difficulty: G1 tasks often have strong lexical overlap between the query and the target tool description, while G3 tasks require cross-category reasoning where the semantic gap is widest. This explains why advanced retrieval strategies (particularly Memetic) show their largest gains on G2 and G3 subsets in Table 2—simple query-based retrieval suffices when the task essentially names the tool, but multi-step, ambiguous tasks demand adaptive search.

A.4 Full End-to-End Results on StableToolBench

Table 3 reports the complete pass rate results across all strategy variants and hyperparameter configurations evaluated on StableToolBench. The main paper (Table 2) reports only the best-performing variant per strategy family; this table includes all ablations.

Key observations: (i) Among Multi-Turn variants, description-by-description refinement (DBD) with T=3 turns outperforms T=5, suggesting diminishing returns from additional iterations. (ii) Iteration-by-iteration refinement (IBI) shows high variance depending on the judge configuration; IBI with judge (j=1) improves G1-Cat but degrades G1-Tool, indicating sensitivity to the fitness signal. (iii) Scattershot with S=5 samples outperforms S=10 on most subsets, suggesting that diversity saturates quickly. (iv) The Memetic strategy (P=5, G=3) dominates all other methods by a wide margin, achieving the highest pass rate on every subset.

Model capability as a prerequisite for evolutionary search. Table 4 reveals a striking reversal: with Qwen3-30B, Memetic (32.2% avg) is the *worst* strategy—below even static Query Retrieval (34.8%)—while with GPT-4.1-mini it is the best (73.4%). Simpler strategies fare better: DBD T=3 leads (47.7%), and Single-Pass is competitive (45.8%). Two patterns emerge. First, **deeper refinement degrades performance**: DBD T=5 (39.7%) $<$ DBD T=3 (47.7%), and Scattershot S=10 (43.3%) $<$ S=5 (45.1%), suggesting that each additional LLM

Table 3: Full pass rate (%) on StableToolBench across all strategy variants. All dynamic methods use GPT-4.1-mini with SimCSE-RoBERTa-large embeddings. Best per column in **bold**. IBI variants were not evaluated on G2/G3 categories due to prohibitively high variance across runs, rendering results unreliable.

Family	Variant	G1-C	G1-I	G1-T	G2-C	G2-I	G3-I	Avg.
Static	Query Retrieval	56.4	52.6	55.6	42.3	47.8	42.1	49.47
Single-Pass	—	61.2	59.0	59.7	55.4	56.5	50.8	57.10
Multi-Turn (DBD)	T=3	69.3	60.5	59.0	50.5	51.3	48.1	56.45
	T=5	67.5	60.0	61.3	55.1	51.6	32.5	54.67
Multi-Turn (IBI)	T=3, j=0	66.0	64.8	55.8	—	—	—	62.20
	T=3, j=1	70.6	63.0	51.4	—	—	—	61.67
	T=5, j=0	64.5	60.3	13.7	—	—	—	46.17
	T=5, j=1	65.1	59.9	42.5	—	—	—	55.83
Scattershot	S=5	62.6	68.7	60.5	55.8	55.0	52.7	59.22
	S=10	64.7	58.9	60.8	61.6	56.3	47.8	58.35
Memetic	P=5, G=3	**77.5**	**72.4**	**68.8**	**70.0**	**76.4**	**75.4**	**73.42**
	V2, P=8, G=3	61.1	62.2	64.3	58.2	57.2	56.6	59.93

Table 4: Pass rate (%) on StableToolBench using **Qwen3-30B** (local, MoE architecture) as the base model for all strategies. Unlike GPT-4.1-mini (Table 3), Memetic *underperforms* simpler strategies, suggesting that evolutionary operators require a sufficiently capable LLM to serve as a semantic operator. Best per column in **bold**.

Family	Variant	G1-C	G1-I	G1-T	G2-C	G2-I	G3-I	Avg.
Static	Query Retrieval	47.9	30.7	32.3	41.9	39.3	16.7	34.80
Single-Pass	—	**58.0**	46.6	41.9	47.4	45.3	35.8	45.80
Multi-Turn (DBD)	T=3	51.4	**49.3**	39.3	**58.2**	37.7	**50.0**	**47.70**
	T=5	52.2	39.5	36.0	46.4	30.7	33.6	39.70
Scattershot	S=5	58.5	47.9	32.0	53.6	36.0	42.3	45.10
	S=10	49.6	36.8	40.9	50.9	**43.7**	38.0	43.30
Memetic	P=5, G=3	42.5	31.7	27.8	38.3	36.0	16.7	32.20

call introduces more noise than signal. Second, **evolutionary operators actively harm quality** when the LLM cannot perform semantically meaningful crossover and mutation—the generated offspring are less coherent than the parents, and selection pressure amplifies these errors across generations. This is consistent with the observation that smaller models struggle with environmental comprehension in agentic settings (Cuadron et al., 2025), and underscores that Memetic's gains are contingent on the base model's capacity to act as a competent semantic operator (Novikov et al., 2025; Guo et al., 2025; Agrawal et al., 2025).

A.5 Per-Subset Retrieval Results for Memetic

Tables 5–7 report per-subset retrieval metrics for *Memetic* across the three ToolRet domains where results are available. These complement the domain-level aggregates in Table 1.

B Method Details

B.1 DFSDT Integration Details

All retrieval strategies in `FitText` operate within the Depth-First Search Decision Tree (DFSDT) reasoning loop (Qin et al., 2024b). Pseudo-tool generation hooks into the DFSDT reasoning chain as follows: when the LLM generates a thought node containing pseudo-tool

Table 5: Memetic per-subset results on ToolRet **Code** domain. Global metrics use merged qrels.

		GPT-4.1-mini				Qwen3-30B			
Subset	**#Q**	N@5	P@5	R@5	C@5	N@5	P@5	R@5	C@5
gorilla-pytorch	43	11.14	3.72	18.61	18.61	8.44	1.86	9.30	9.30
gorilla-tensor	55	3.10	1.45	7.27	7.27	3.31	1.09	5.45	5.45
gorilla-huggingface	500	24.15	6.52	32.60	32.60	23.23	6.96	34.80	34.80
craft-tabmwp	174	12.92	3.79	18.97	18.97	15.19	4.14	20.69	20.69
craft-vqa	200	11.52	2.90	14.50	14.50	12.51	2.90	14.50	14.50
craft-math-algebra	280	64.29	14.64	73.21	73.21	60.31	14.07	70.36	70.36
toolink	497	52.80	21.05	54.52	41.25	54.23	20.81	55.64	41.45
Global	1749	**35.18**	**11.04**	**40.76**	**36.99**	**34.96**	**10.98**	**41.14**	**37.11**

Table 6: Memetic per-subset results on ToolRet **Customized** domain.

		GPT-4.1-mini				Qwen3-30B			
Subset	**#Q**	N@5	P@5	R@5	C@5	N@5	P@5	R@5	C@5
gpt4tools	32	60.32	14.38	71.88	71.88	55.87	14.38	71.88	71.88
taskbench-hf	23	42.08	21.74	47.07	30.43	26.87	16.52	37.46	21.74
taskbench-mm	40	73.07	34.00	75.02	55.00	65.72	32.00	72.33	52.50
toolbench-sam	197	7.49	1.93	9.65	9.65	8.32	2.44	12.18	12.18
toolalpaca	94	54.80	13.62	68.09	68.09	59.41	14.47	72.34	72.34
gta	14	13.14	4.29	10.71	0.00	29.04	8.57	25.00	7.14
tool-be-honest	350	35.88	25.94	31.11	0.00	33.05	25.14	29.67	0.00
appbench	32	75.64	32.50	82.55	59.38	71.45	28.75	72.14	50.00
metatool	200	37.47	9.30	44.50	42.50	29.60	7.50	35.50	34.50
Global	982	**35.75**	**16.31**	**37.95**	**24.34**	**33.03**	**15.58**	**36.05**	**23.12**

description blocks, the system triggers a dynamic retrieval call. The choice of retrieval strategy (Single-Pass, IBI, DBD, Scattershot, or Memetic) is determined by the configured policy. Retrieved tools are injected into the active function schema for subsequent action nodes.

Notation. Let $\mathcal{N}$ denote the set of tree nodes, where each node n carries a message history $\mathcal{H}(n)$, type $\tau(n) \in \{\text{Thought, Action, ActionInput}\}$, depth $\delta(n)$, and child set $\text{ch}(n)$. Let $\mathcal{F}(n)$ be the function schema (tool list) active at node n, and let $\textsc{Exec}(a, x, \mathcal{F})$ execute action a with arguments x against the API environment, returning an observation and status code $\kappa \in \{0{=}\text{ok},\ 1{=}\text{halluc},\ 2{=}\text{error},\ 3{=}\text{answer},\ 4{=}\text{give\_up}\}$. We write $D_{\max}$ for the maximum branch depth, W for beam width, $Q_{\max}$ for the LLM call budget, and $\ell_{\text{prune}}, \ell_{\text{ans}}$ for backtrack distances.

Mathematical formalization. Let the search tree be $\mathcal{G} = (\mathcal{N}, E)$ where each edge $(n, c) \in E$ corresponds to one LLM generation step. (We use $\mathcal{G}$ to avoid overloading $\mathcal{T}$ which denotes the tool corpus.) The message history at node n along path $\pi(n) = (n_0, n_1, \ldots, n)$ is:

$$\mathcal{H}(n) = \big[\text{sys}(q)\big] \circ \big[\text{usr}(q)\big] \circ \big(a_i, x_i, o_i\big)_{i \in \pi(n)}, \tag{8}$$

where (a_i, x_i, o_i) are action/argument/observation triples. Backtracking propagates when $b > 1$, continuing expansion only when $b \leq 1$. Dynamic retrieval triggers when:

$$\max_{m \in \mathcal{M}} \text{SequenceSim}(o, m) < 0.82 \ \wedge\ o \not\subset m \ \ \forall m \in \mathcal{M}. \tag{9}$$

B.2 Strategy Details: Scattershot, DBD, and IBI

We detail the three non-evolutionary strategies below. All use the shared notation: q (query), $\mathcal{T}$ (tool corpus), $\mathbf{e}(\cdot)$ (sentence embedding via SimCSE-RoBERTa-large), $\textsc{Ret}(d, \mathcal{T}, k)$ (top-k

Table 7: Memetic per-subset results on ToolRet **Web** domain (19 subsets, 5,230 queries).

		GPT-4.1-mini				Qwen3-30B			
Subset	**#Q**	N@5	P@5	R@5	C@5	N@5	P@5	R@5	C@5
autotools-weather	11	1.33	1.82	1.82	0.00	4.47	3.64	5.30	0.00
autotools-food	22	33.25	14.55	34.85	13.64	32.18	18.18	43.18	18.18
autotools-music	32	8.35	7.50	9.32	0.00	6.40	5.00	6.09	0.00
restgpt-spotify	40	24.37	10.00	24.58	15.00	26.99	11.50	25.63	12.50
restgpt-tmdb	54	5.93	2.96	6.48	0.00	12.31	4.81	11.11	0.00
toolemu	38	11.20	3.68	15.79	13.16	12.14	3.16	13.16	10.53
mnms	33	25.28	12.73	26.77	6.06	16.38	9.70	19.19	6.06
taskbench-daily	40	27.01	11.50	27.75	15.00	11.98	6.50	17.04	12.50
t-eval-dialog	50	30.58	8.40	42.00	42.00	27.70	8.80	44.00	44.00
t-eval-step	50	35.64	21.60	34.82	8.00	31.22	19.60	31.71	8.00
tooleyes	95	16.57	4.63	23.16	23.16	17.44	4.84	24.21	24.21
apibank	101	23.51	7.72	26.73	16.83	24.34	7.33	26.65	18.81
reversechain	200	19.90	11.40	19.88	2.50	13.94	7.80	13.77	1.00
toollens	314	3.95	2.23	4.83	1.27	4.78	2.80	6.10	1.59
rotbench	550	4.99	1.75	8.73	8.73	5.13	1.89	9.36	9.27
ultratool	500	46.78	19.88	50.89	31.20	40.70	17.72	45.62	27.40
apigen	1000	44.28	13.32	54.55	49.60	42.84	13.02	53.08	49.10
toolace	1000	54.20	16.06	62.61	58.30	48.46	14.52	57.98	54.30
toolbench	1100	32.05	15.71	35.48	17.09	31.51	15.53	35.12	16.45
Global	5230	**33.84**	**12.51**	**39.33**	**29.94**	**31.49**	**11.78**	**37.43**	**28.64**

retrieval by cosine similarity), blurb(t) (normalized natural-language description of tool t), and $J(A, B) = |A \cap B|/|A \cup B|$ (Jaccard similarity).

DBD (Description-by-Description). DBD maintains independent per-lineage refinement paths, each anchored to its original ancestor description. Unlike IBI, exemplars are lineage-local and the ancestor is included in every refinement prompt as an anchor.

For each ancestor $a \in \mathcal{D}_0$, define $d_a^{(0)} = a$ and $\mathcal{E}_a^{(0)} = \emptyset$. At turn t:

$$\mathcal{E}_a^{(t)} = \mathcal{E}_a^{(t-1)} \cup \{\text{blurb}(r) : r \in \textsc{Ret}(d_a^{(t-1)}, \mathcal{T}, k)\}, \tag{10}$$

$$d_a^{(t)} = \textsc{ParseSingle}\Big(\text{LLM}_{\text{refine}}(a, d_a^{(t-1)}, \mathcal{E}_a^{(t)}, q)\Big). \tag{11}$$

Final output: Output $= \bigcup_{a \in \mathcal{D}_0} \textsc{Ret}(d_a^{(n_t)}, \mathcal{T}, k)$. LLM cost per lineage: n_t calls; retrievals: $n_t + 1$.

IBI (Iterative Bootstrapped Improvement). IBI applies a single global refinement prompt across all descriptions simultaneously. It accumulates exemplar tool blurbs across iterations and optionally uses an LLM judge to check sufficiency.

Let $\mathcal{D}^{(0)}$ be the initial description set. At each turn t:

$$\mathcal{B}^{(t)} = \mathcal{B}^{(t-1)} \cup \bigcup_{d \in \mathcal{D}^{(t-1)}} \{\text{blurb}(r) : r \in \textsc{Ret}(d, \mathcal{T}, k)\}, \tag{12}$$

$$\mathcal{D}^{(t)} = \textsc{ParseBlocks}\Big(\text{LLM}_{\text{refine}}(\mathcal{D}^{(t-1)}, \mathcal{B}^{(t)}, q)\Big). \tag{13}$$

LLM cost (worst case, n_t turns, m descriptions): n_t refinement calls $+n_t$ optional judge calls; retrievals: $(n_t + 1) \cdot m$.

Scattershot. Scattershot introduces population-level diversity by sampling multiple candidate descriptions per ancestor in parallel at high temperature, then aggregating via population-level voting.

Algorithm 2 DFSDT Reasoning Loop with Dynamic Retrieval

Require: Node n, $D_{\max}$, W, $Q_{\max}$, answer set $\mathcal{S}$, memory $\mathcal{M}$, function schema $\mathcal{F}$
Ensure: Backtrack distance b; modifies $\mathcal{S}$, $\mathcal{M}$, $\mathcal{F}$

if $\delta(n) \geq D_{\max}$ or n.pruned **then**
  return ℓ_{prune}
end if
if n.terminal **then**
  $\mathcal{S}$.add(n)
  return ℓ_{ans}
end if
for $i \leftarrow 1$ to W **do**
  if $|\mathcal{S}| \geq \mathcal{S}_{\max}$ or $Q > Q_{\max}$ **then return** ∞
  end if
  if ch$(n) \neq \emptyset$ **then**
    Inject diversity prompt summarising prior children
  end if
  $o \leftarrow \text{LLM}(\mathcal{H}(n), \mathcal{F}(n))$; $Q \leftarrow Q + 1$
  if o contains pseudo-tool blocks **then**
    $n_{\text{th}} \leftarrow$ Thought child
    if not duplicate in $\mathcal{M}$ **then**
      tools $\leftarrow$ SELECTANDRUNSTRATEGY$(o, \mathcal{F}, \text{LLM}, \mathcal{M})$
      $\mathcal{F}(n_{\text{th}}) \leftarrow \mathcal{F}(n_{\text{th}}) \cup$ *tools*
      $\mathcal{M}$.update(o)
    end if
  end if
  for each function call $(a, x) \in o$ **do**
    $n_a \leftarrow$ Action child; $n_x \leftarrow$ ActionInput child
    $(\text{obs}, \kappa) \leftarrow \text{EXEC}(a, x, \mathcal{F}(n))$
    if $\kappa = 1$ **then** replace a with sentinel
    end if
    if $\kappa = 3$ **then** n_x.terminal $\leftarrow$ True
    end if
    if $\kappa \in \{1, 2, 4\}$ **then** n_x.pruned $\leftarrow$ True
    end if
  end for
  for each child $c \in$ ch(n) **do**
    $b \leftarrow \text{DFSDT}(c, D_{\max}, W, Q_{\max}, \mathcal{S}, \mathcal{M}, \mathcal{F})$
    if $|\mathcal{S}| \geq \mathcal{S}_{\max}$ **then return** ∞
    end if
    if $b > 1$ **then return** $b - 1$
    end if
  end for
end for
return 1

The following subroutines are shared between Scattershot and Memetic (Algorithm 1):

For each ancestor a, let $\mathcal{E}_a = \{\text{blurb}(r) : r \in \text{RET}(a, \mathcal{T}, k)\}$ be seed exemplars. Draw a population of S children independently:

$$d_i \sim \text{LLM}_{\text{scatter}}(a, \mathcal{E}_a;\ T_b), \quad i = 1, \ldots, S, \tag{14}$$

where $T_b = 1.5$ is fixed. For each ancestor a, VOTE is applied to $\{d_1, \ldots, d_S\}$. Scattershot is Memetic with G=1 and no fitness evaluation, selection, or local search, a degenerate single-generation case that establishes the population-voting idea. LLM cost per lineage: S parallel calls; retrievals: $S + 1$.

Algorithm 3 DBD — Per-Lineage Iterative Refinement

Require: LLM output o, corpus $\mathcal{T}$, top-k, turns n_t
Ensure: API key list
$\mathcal{D}_0 \leftarrow \textsc{ParseBlocks}(o)$
for each ancestor $a \in \mathcal{D}_0$ **do**
  $d \leftarrow a$; $\mathcal{E} \leftarrow \emptyset$
  for $t \leftarrow 1$ to n_t **do**
    $(Q, R) \leftarrow \textsc{Ret}(d, \mathcal{T}, k)$
    $\mathcal{E} \leftarrow \mathcal{E} \cup \{\text{blurb}(r) : r \in R\}$
    $d' \leftarrow \textsc{ParseSingle}(\text{LLM}_{\text{refine}}(a, d, \mathcal{E}, q))$
    if $d' \neq \emptyset$ **then** $d \leftarrow d'$
    end if
  end for
  Append d to finals
end for
return $\bigcup_{d \in finals} \textsc{Ret}(d, \mathcal{T}, k)$.api_list

Algorithm 4 IBI — Iterative Bootstrapped Improvement

Require: LLM output o, corpus $\mathcal{T}$, top-k, turns n_t, sufficiency flag CHECK
Ensure: API key list
$\mathcal{D} \leftarrow \textsc{ParseBlocks}(o)$; $\mathcal{B} \leftarrow \emptyset$
for $t \leftarrow 1$ to n_t **do**
  for each $d \in \mathcal{D}$ **do**
    $(Q, R) \leftarrow \textsc{Ret}(d, \mathcal{T}, k)$
    $\mathcal{B} \leftarrow \mathcal{B} \cup \{\text{blurb}(r) : r \in R\}$
  end for
  if CHECK and $\text{LLM}_{\text{judge}}(q, \mathcal{B}) = \text{True}$ **then**
    break
  end if
  $o' \leftarrow \text{LLM}_{\text{refine}}(\mathcal{D}, \mathcal{B}, q)$
  $\mathcal{D}' \leftarrow \textsc{ParseBlocks}(o')$
  if $\mathcal{D}' = \emptyset$ **then break**
  end if
  $\mathcal{D} \leftarrow \mathcal{D}'$
end for
for each $d \in \mathcal{D}$ **do**
  Append $\textsc{Ret}(d, \mathcal{T}, k)$.api_list to keys
end for
return *keys*

B.3 Hyperparameter Settings and Notation

Table 8 lists key hyperparameters. Table 9 summarizes the mathematical notation.

Temperature note. A temperature of $\tau = 1.5$ for GPT-4.1-mini is critical for memetic performance. Higher temperature promotes solution divergence in the initial population, preventing premature convergence during the exploration phase of evolutionary search.

C Memetic Extensions

C.1 Memetic V2 Algorithm

Algorithm 8 presents the full Memetic V2 pseudocode. V2 extends V1 with an LLM-based scoring component in the fitness function, temperature annealing, tournament selection, adaptive crossover rate, and guided mutation. Below we detail each component.

Algorithm 5 Scattershot — Parallel Diversity Sampling

Require: LLM output o, corpus $\mathcal{T}$, top-k, fan-out size S, temperature T_b, budget B
Ensure: API key list
$\mathcal{D}_0 \leftarrow \textsc{ParseBlocks}(o)$
for each ancestor $a \in \mathcal{D}_0$ **do**
  $(Q_0, R_0) \leftarrow \textsc{Ret}(a, \mathcal{T}, k)$
  $\mathcal{E} \leftarrow \{\text{blurb}(r) : r \in R_0\}$
  $C \leftarrow \textsc{Parallel}[\text{LLM}_{\text{scatter}}(a, \mathcal{E};\ T_b)]_{i=1}^{S}$
  $\mathcal{P} \leftarrow \{c_i : c_i \leftarrow \textsc{ParseSingle}(C_i),\ c_i \neq \emptyset\}$
  Extend keys with $\textsc{Vote}(\mathcal{P}, \mathcal{T}, k, B)$
end for
return *keys*

Algorithm 6 Seed Population

Require: Ancestor a, $\mathcal{T}$, k, N, τ
Ensure: Population $\mathcal{P}$
$R_0 \leftarrow \textsc{Ret}(a, \mathcal{T}, k)$
$\mathcal{E} \leftarrow \{\text{blurb}(r) : r \in R_0\}$
$\mathcal{P} \leftarrow \text{LLM}_{\text{seed}}(a, \mathcal{E}, N;\ \tau)$
return $\mathcal{P}$

Fitness function f_2.

$$f_2(d_i) = \alpha\, s_{\text{ret}}(d_i, R_i) + \beta\, \text{LLM}_{\text{score}}(q, R_i) - \gamma\, \text{mem}(d_i) - \delta\, \text{crowd}(d_i), \tag{15}$$

where default weights are $(\alpha, \beta, \gamma, \delta) = (0.5, 0.3, 0.2, 1.0)$ and $s_{\text{ret}}(d_i, R_i) = 0.7\,\sigma_1(R_i) + 0.3\,\bar{\sigma}_3(R_i)$, with σ_1 the top-1 and $\bar{\sigma}_3$ the mean top-3 cosine similarity scores from retrieval $R_i = \textsc{Ret}(d_i, \mathcal{T}, k)$. The crowding term uses $\delta = 1.0$ (unit weight) by design, since crowd $\in [0, 1]$ is already normalized to the same scale.

Note that LLM-generation of a population operates at the *semantic* level, while Jaccard similarity operates at the *syntactic* level; together they provide complementary diversity signals that prevent the population from collapsing to a single mode.

Temperature annealing.

$$T_g = \max(T_b - 0.4,\ \ T_b + 0.1 - 0.15\, g)\,, \tag{16}$$

with mutation receiving an additional $+0.1$ bonus. With $T_b = 1.5$: $T_0 = 1.6$, $T_1 = 1.45$, $T_2 = 1.30$, $T_3 = 1.15$ (floor at $T_b - 0.4 = 1.1$).

Memory penalty with exponential decay.

$$\text{mem}(d_i) = \max_{(M_j, g_j) \in \mathcal{M}} \left[0.5 \cdot J(R_i, M_j) \cdot e^{-g_j/3}\right]. \tag{17}$$

Newer memories (g_j small) penalize more heavily; older memories decay exponentially.

Crowding penalty.

$$\text{crowd}(d_i) = \frac{1}{N-1} \sum_{j \neq i} \mathbb{1}\left[\cos(\mathbf{e}(d_i), \mathbf{e}(d_j)) > \theta\right]. \tag{18}$$

For the edge case $N = 1$, define $\text{crowd}(d_i) = 0$.

Tournament selection.

$$\text{select}() = \arg\max_{d \in S}\ f_2(d), \qquad S \sim \text{Uniform}(\mathcal{P}, 3). \tag{19}$$

Algorithm 7 Hierarchical Voting

Require: Population $\mathcal{P}$, $\mathcal{T}$, k, budget B
Ensure: Top-B ranked tool list
for each $d_i \in \mathcal{P}$ **do**
  $R_i \leftarrow \textsc{Ret}(d_i, \mathcal{T}, k)$
end for
$\mathcal{C} \leftarrow \bigcup_i R_i$ *candidate pool*
for each tool $t \in \mathcal{C}$ **do**
  $v(t) \leftarrow |\{i : t \in R_i\}|$ *vote count*
  $\bar{r}(t) \leftarrow \text{mean}_{i:t\in R_i} \text{rank}_i(t)$
  $\bar{s}(t) \leftarrow \text{mean}_{i:t\in R_i} \text{sim}_i(t)$
end for
Sort $\mathcal{C}$ by $(v\downarrow,\ \bar{r}\uparrow,\ \bar{s}\downarrow)$
return $\text{top}_B(\mathcal{C})$

Table 8: Hyperparameter settings for all experiments.

Parameter	**Description**	**Value**
k	Top-k retrieval candidates	5
$\mathbf{e}(\cdot)$	Embedding model	SimCSE-RoBERTa-large
Analysis LLM	Query analysis model	GPT-4.1
Refinement LLM	Description refinement model	GPT-4.1-mini / Qwen3-30B
T	Multi-Turn refinement turns	3 or 5
τ_{refine}	Multi-Turn temperature	≤ 0.7
S	Scattershot fan-out size	5 or 10
τ_{scatter}	Scattershot generation temperature	1.5
N	Memetic population size	6 (V1) / 8 (V2)
G	Memetic generations	3
θ	Memetic fitness threshold	0.95
θ_c	Crowding similarity threshold	0.85
τ_{memetic}	Memetic base temperature	1.5
B	Voting budget (top-B output)	5
Tool corpus sizes		
StableToolBench	Total tool corpus	16,464 APIs
ToolRet	Total tool corpus	43k tools

Adaptive crossover rate.

$$p_{\text{cross}}(g) = \max(0.4,\ 0.6 - 0.1\, g). \tag{20}$$

Guided mutation. Let $\mathcal{L}(d) = \text{bottom}_m\{R_d\}$ be the m lowest-scoring tools retrieved by d:

$$d_{\text{child}} \leftarrow \begin{cases} \text{LLM}_{\text{guided}}(d_c, \mathcal{L}(d_a);\ T_g + 0.1) & \text{if low-quality tools detected} \\ \text{LLM}_{\text{mut}}(d_c;\ T_g + 0.1) & \text{otherwise} \end{cases} \tag{21}$$

Table 9: Summary of mathematical notation.

Symbol	Definition
q	User query
$a_t \in \mathcal{A}$	Action at time step t
$\mathcal{A}$	Action space
$\mathcal{F}_t \subseteq \mathcal{T}$	Function schema (active tools) at step t
$\mathcal{T} = \{t_1, \ldots, t_n\}$	Tool corpus
d	Pseudo-tool description
$\mathcal{D}_0$	Initial pseudo-tool descriptions (ancestors)
$\mathcal{H}$	Message history
$\mathbf{e}(\cdot) \to \mathbb{R}^d$	Sentence embedding function
$R \subseteq \mathcal{T}$	Retrieved tool set (top-k)
$\mathcal{E}$	Exemplar set (tool blurbs)
$f(d_i) \in \mathbb{R}$	Fitness function
$\mathcal{P}$	Population of descriptions
$\mathcal{M}$	Tool memory
$\mathcal{S}$	Answer set (DFSDT)
$\tau \in \mathbb{R}^+$	Temperature
$\theta \in [0, 1]$	Fitness threshold
$N \in \mathbb{N}$	Population size
$G \in \mathbb{N}$	Number of generations
$B \in \mathbb{N}$	Retrieval budget

Algorithm 8 Memetic V2

Require: LLM output o, corpus $\mathcal{T}$, top-k, memory $\mathcal{M}$, N=8, G=3, threshold θ=0.95, weights α, β, γ, temperature T_b=1.5, budget B
Ensure: API key list; $\mathcal{M}$ updated

$\mathcal{D}_0 \leftarrow \textsc{ParseBlocks}(o)$
for each ancestor $a \in \mathcal{D}_0$ **do**
  $(Q_0, R_0) \leftarrow \textsc{Ret}(a, \mathcal{T}, k)$; $\mathcal{E} \leftarrow \{\text{blurb}(r) : r \in R_0\}$
  $\mathcal{P} \leftarrow \text{LLM}_{\text{seed}}(a, \mathcal{E}, N;\ T_0)$
  for $g \leftarrow 1$ to G **do**
    $T_g \leftarrow$ temperature from Eq. 16
    for each $d_i \in \mathcal{P}$ **do**
      $R_i \leftarrow \textsc{Ret}(d_i, \mathcal{T}, k)$; $f_i \leftarrow f_2(d_i)$ via Eq. 15
    end for
    $d^* \leftarrow \arg\max_i f_i$; $f^* \leftarrow \max_i f_i$
    if $f^* \geq \theta$ or $g = G$ **then break**
    end if
    $\mathcal{P}_{\text{sel}} \leftarrow \emptyset$
    while $|\mathcal{P}_{\text{sel}}| < \lceil N/2 \rceil$ **do**
      $S \leftarrow \text{sample}(\mathcal{P}, 3)$; $\mathcal{P}_{\text{sel}} \leftarrow \mathcal{P}_{\text{sel}} \cup \{\arg\max_{d \in S} f_2(d)\}$
    end while
    $\mathcal{P}' \leftarrow \{d^*\}$
    while $|\mathcal{P}'| < N$ **do**
      Draw $u \sim \text{Bernoulli}(p_{\text{cross}}(g))$
      if $u = 1$ **then**
        $d_a, d_b \leftarrow \text{sample}(\mathcal{P}_{\text{sel}}, 2)$
        $d_c \leftarrow \textsc{ParseSingle}(\text{LLM}_{\text{cross}}(d_a, d_b, a;\ T_g))$
      else
        $d_a \leftarrow \text{sample}(\mathcal{P}_{\text{sel}}, 1)$
        $d_c \leftarrow \textsc{ParseSingle}(\text{LLM}_{\text{guided or mut}}(d_a;\ T_g + 0.1))$
      end if
      $\mathcal{P}' \leftarrow \mathcal{P}' \cup \{d_c\}$
    end while
    for each $d \in \mathcal{P}' \setminus \{d^*\}$ **do**
      $R_d \leftarrow \textsc{Ret}(d, \mathcal{T}, k)$; $\mathcal{E}_d \leftarrow \{\text{blurb}(r) : r \in R_d\}$
      $d \leftarrow \textsc{ParseSingle}(\text{LLM}_{\text{refine}}(a, d, \mathcal{E}_d, q;\ T_g))$
    end for
    $\mathcal{P} \leftarrow \mathcal{P}'$
  end for
  $\mathcal{M} \leftarrow \mathcal{M} \cup \{(a, G),\ (d^*, 0)\}$
  Extend keys with $\textsc{Vote}(\mathcal{P}, \mathcal{T}, k, B)$
end for

C.2 Tool Memory Mechanism

Tool memory $\mathcal{M}$ is a shared dictionary mapping pseudo-tool intent strings to generation indices. It persists across all branches of a single DFSDT invocation and, within evolutionary strategies, across generations. The interaction with the tool database can be understood as *tool exploration*: the database is large and heterogeneous, so previously encountered information guides the search using imperfect signals.

Jaccard-based redundancy penalty. The memory penalty (Eq. 17) uses Jaccard similarity between the current retrieval set R_i and each stored set M_j, weighted by exponential staleness decay $e^{-g_j/3}$. This ensures that recently explored tool regions are penalized more heavily, encouraging the population to explore new areas of the tool corpus.

Write points.

1. **After non-evolutionary retrieval** (Single-Pass, IBI, DBD): all pseudo-tool descriptions from the completed call are added to $\mathcal{M}$.
2. **After evolutionary finalization**: the ancestor a and the best individual d^* are added. V1 stores the current generation counter g for both; V2 stores $g_j = G$ for the ancestor and $g_j = 0$ for d^*, so that the freshest entry is always penalized most heavily.

Read points.

1. **DFSDT duplicate check** (Algorithm 2): suppresses redundant retrieval across sibling branches using SequenceMatcher similarity ≥ 0.82.
2. **Fitness evaluation** (Equations 15 and 17): penalizes descriptions whose retrieval results overlap too heavily with previously explored intents.

C.3 Evolutionary Extensions and Framework

`FitText` draws on ideas from evolutionary computation, particularly memetic algorithms (Guo et al., 2025). Table 10 maps `FitText` concepts to their evolutionary counterparts.

Table 10: Mapping between `FitText` concepts and evolutionary computation terminology.

`FitText` Concept	Evolutionary Term
Pseudo-tool description	Meme / Gene
Set of candidate descriptions	Population
LLM refinement	Mutation
Retrieval quality score	Fitness
Tool database	Environment
Population-level voting	Selection
Multi-turn feedback (DBD operator)	Local search
Scattershot diverse generation	Exploration

Temperature importance. We use temperature $\tau = 1.5$ for GPT-4.1-mini to ensure sufficient diversity in the generated population. Higher temperature promotes solution divergence (Li et al., 2025a), which is critical for the exploration phase of evolutionary search—Yang et al. (2024) observe that high temperature "allows the LLM to more aggressively explore solutions that can be notably different," directly motivating our choice. Without sufficient initial diversity, the population converges prematurely and the evolutionary operators (crossover, mutation, selection) have limited material to work with.

C.4 Strategy Design Space

Table 11 compares the key design dimensions across all `FitText` strategies.

Table 11: Strategy design space comparison. LLM cost level is relative per lineage.

Strategy	Iterative	Indep. lineages	Diverse gen.	Fitness-driven	Pop. voting	LLM cost
Single-Pass	No	—	No	No	No	None
IBI	Yes	No	No	No	No	Low
DBD	Yes	Yes	No	No	No	Low
Scattershot	No	Yes	Yes	No	Yes	Medium
Memetic	Yes	Yes	Yes	Yes	Yes	High
Memetic V2	Yes	Yes	Yes	Yes	Yes	Highest

D Discussion: Conceptual Perspective

Beyond the empirical results, `FitText` introduces several conceptual shifts in how tool retrieval is framed. We briefly discuss four that we believe are relevant to future work in this area.

Pseudo-tools as information artifacts. In our framework, pseudo-tool descriptions are not static queries but *structured semantic hypotheses* that evolve under selection pressure. Each description encodes the agent's belief about what tool it needs, and this belief is iteratively refined through interaction with the tool database. This view—tools as mutable information artifacts rather than fixed retrieval targets—opens connections to hypothesis-driven search in scientific discovery and active learning. Notably, pseudo-tool generation implicitly interleaves planning and execution: the agent plans what it needs by describing it, diverging from the traditional plan-then-execute decomposition (Huang et al., 2024).

The tool ecology metaphor. We frame the API catalog as an ecology in which tools occupy functional niches defined by user queries (§4.1). Pseudo-tool descriptions act as organisms competing to fill these niches, with retrieval quality serving as fitness. This framing naturally explains why population diversity matters (Scattershot) and why selection pressure improves over random exploration (Memetic): ecological competition rewards both breadth and adaptation.

Tool chain emergence. In multi-step tasks, the agent generates multiple pseudo-tool descriptions that co-evolve across retrieval episodes. When these descriptions are refined in parallel through the memetic process, they can develop complementary specializations—one description targets data retrieval while another targets data transformation—forming emergent tool chains without explicit orchestration. This mirrors memeplex formation in cultural evolution, where co-adapted ideas propagate together.

Connections to dynamical systems. Viewing tool ecosystems as populations under selection implicitly invokes a dynamical-systems perspective: pseudo-tool descriptions occupy positions on a fitness landscape defined by retrieval quality, and the evolutionary operators (selection, crossover, mutation) induce population dynamics analogous to replicator equations. While we do not formalize this connection, we note that concepts such as fitness landscape ruggedness (explaining why Single-Pass gets stuck) and drift (explaining stochastic variation across Scattershot runs) may provide a useful theoretical vocabulary for future analysis of pseudo-tool search dynamics.

Intersection of multi-hop QA and information retrieval. Tool-augmented agents operating in the open domain lie at the intersection of multi-hop question answering and information retrieval. The resulting system must reason over disparate knowledge sources, reconcile conflicting information from different tools, and manage retrieval-induced uncertainty—all while maintaining coherent multi-step decision-making. Our framework addresses this by treating each retrieval episode as an independent search problem whose results feed back into the agent's evolving reasoning state.

Algorithm stability and lineage tracking. The choice between IBI (global refinement) and DBD (per-lineage refinement) has significant implications for stability. IBI lacks pseudo-tool provenance tracking: at each iteration, a variable number of descriptions may be generated, making it difficult to attribute improvements to specific lineages. DBD addresses this by maintaining independent refinement paths with explicit lineage anchors, yielding more stable convergence. This design choice—seemingly minor—proved critical in practice and underscores the importance of algorithm formulation in evolutionary tool search.

Semantic alignment inheritance. In Scattershot and Memetic strategies, the ancestor pseudo-tool description has already undergone semantic alignment through the LLM's initial generation step. Its progeny inherit this alignment benefit: diversified variants operate in a semantically richer region of description space than raw user queries, giving the entire population a head start in retrieval quality.

ToolBench as a reference coordinate system. ToolBench (Qin et al., 2024b) and its stabilized variant StableToolBench (Guo et al., 2024) have become de facto benchmarks for tool learning, with subsequent work increasingly positioning itself relative to ToolBench's assumptions—whether by addressing its API instability, scaling its tool catalog, or relaxing its closed-domain setup. Our work contributes to this ecosystem by demonstrating that the retrieval interface, largely abstracted away in prior ToolBench studies, is itself a critical and improvable component of the agent pipeline.

E Prompt Templates

We provide the key LLM prompt templates used across the `FitText` pipeline. Template variables are shown as {`variable`}. The special tokens {`BEGIN`} and {`END`} delimit pseudo-tool description blocks that trigger retrieval.

Dynamic Retrieval Protocol. The following preamble is prepended to the agent's system prompt in open-domain mode, instructing the agent to use the {`BEGIN`}...{`END`} protocol for on-demand tool retrieval:

```
You should use functions to help handle the real time user queries.
1. Dynamic Function Retrieval: The only function available at the start
   is "Finish". To call any other function, you must first retrieve it
   from a function corpus based on your needs.
2. Retrieval Process: To retrieve the required function, write:
   {BEGIN} [Describe the function you need in detail] {END}
   If you need multiple functions, describe each separately using its
   own {BEGIN}...{END} block. Once written, pause and wait until the
   system retrieves the available functions for you.
3. Task Completion: You must always call "Finish" at the end. The final
   output must be self-contained and complete.
```

DBD Refinement. Given the current pseudo-tool description, the original ancestor, and exemplar tools from prior retrievals, the LLM refines the description to better match the tool database:

```
You are assisting an autonomous agent that has already decomposed the
task into multiple subtasks. Your role is to refine exactly one function
description for the current subtask only.
Context:
- Global user query: {query}
- Draft function description to refine: {current_desc}
- Retrieved function examples from the database: {lineage_examples}
Goal: Refine the description so it (1) closely matches the style and
structure of the retrieved examples, and (2) is optimized for retrieval.
Output: Exactly one {BEGIN}...{END} block. No commentary.
```

Scattershot Diversification. Each call generates one diversified variant; the caller invokes this S times in parallel to build a diverse candidate pool:

```
Your role is to generate exactly one diversified function description.
Context:
- Global user query: {query}
- Draft description to diversify (reference only): {ancestor_desc}
- Retrieved function examples: {exemplar_block}
Goal: Generate a new description that (1) preserves the subtask intent,
(2) follows the style of retrieved examples, (3) is optimized for
retrieval, and (4) introduces variation in phrasing or emphasis.
Output: Exactly one {BEGIN}...{END} block. Vary wording each call.
```

Memetic Population Seeding. Generates the initial population of N diverse pseudo-tool descriptions from a single ancestor:

```
You must propose diversified pseudo-tools for the SAME subtask so a
retriever can search for better-matching tools.
Context:
- Global user query: {query}
- Ancestor pseudo-tool (keep its intent): {ancestor_desc}
- Retrieved exemplar descriptions: {exemplar_block}
Goal: Produce {population_size} alternative pseudo-tool descriptions
that preserve the ancestor intent but vary wording and structure.
Output: Exactly {population_size} {BEGIN}...{END} blocks. No extra text.
```

Memetic Crossover. Combines two parent descriptions into one child:

```
Create exactly one coherent child description that combines the
strongest constraints, parameters, and intent from both parents.
- Parent A: {parent1_desc}
- Parent B: {parent2_desc}
- Lineage anchor: {ancestor_desc}
Output: Exactly one {BEGIN}...{END} block. No commentary.
```

Memetic Mutation. Produces one mutated variant while preserving intent:

```
Create exactly one mutated variant of the parent that keeps the same
intent but adjusts wording/constraints to improve retrieval.
- Parent description: {parent_desc}
- Lineage anchor: {ancestor_desc}
Output: Exactly one {BEGIN}...{END} block. No commentary.
```

Memetic Guided Mutation. Steers the mutation away from low-fitness retrieved tools:

```
Create one mutated variant that keeps the same intent but steers AWAY
from the low-scoring tools listed below.
- Parent description: {parent_desc}
- Low-scoring tools to avoid: {low_scoring_tools}
Modify wording to AVOID retrieving the low-scoring tools. Shift
emphasis toward aspects those tools failed to address.
Output: Exactly one {BEGIN}...{END} block. No commentary.
```

Memetic Fitness Judge. Scores (0.0–1.0) how well retrieved tools cover the user query:

```
Rate how well these retrieved tools collectively address the user query:
- User query: {query}
- Pseudo-tool used for retrieval: {pseudotool}
- Retrieved tools: {retrieved_tools_block}
Scale: 1.0=perfect, 0.7=mostly covers, 0.4=partial, 0.0=irrelevant
Output: ONLY a single decimal number between 0.0 and 1.0.
```